

1. Del Cliente al Servidor 🖥️ → 🖥️

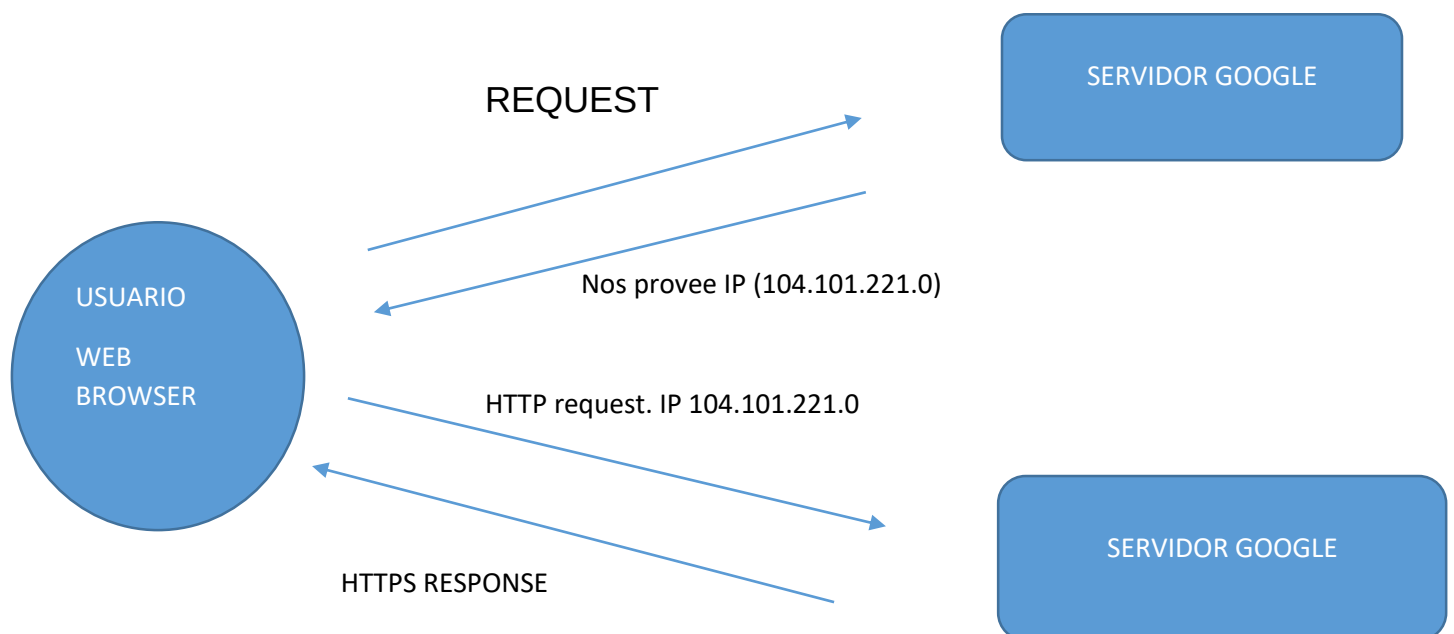
- Explique paso a paso qué ocurre cuando usted escribe `www.youtube.com` en el navegador hasta que el video aparece en pantalla.

- Debe incluir en su explicación:

- Qué hace el cliente (navegador).
- Qué papel cumple el DNS.
- Qué ocurre con la dirección IP.
- Qué hace el servidor.
- Cómo entra en juego el protocolo HTTP/HTTPS.
-

1. Explicación:

- Nuestro navegador web (cliente) **solicita** o hace un request al servidor de Google usando Domain name system, servidores encargados de **traducir** los nombres de dominio URL a dirección IP (protocolo de internet) publicas estas **conectan** al servidor con datos,
- El servidor de Google nos da un response, que HTTP hyper text tiping protocol, **transmite**. En información.
- El http es el protocolo estándar para enviar el request y recibir el response, datos entre cliente y servidor.



2. Frontend y Backend en acción ☐

- Suponga que está construyendo una app web para agendar citas médicas.

- Indique qué parte del sistema correspondería al frontend y cuál al backend.
- Mencione tres tecnologías posibles para cada uno.
- Explique brevemente cómo el frontend se comunicaría con el backend (mencione los conceptos de API, HTTP y request/response).

Supongamos que tenemos una aplicación para agendar citas médicas como él (EDUS) en Costa Rica.

La parte que le corresponde al área del Frontend es la parte visible para el usuario de la aplicación,

Lo que muestra el navegador, Formularios de registro, iniciar sesión y agendar cita, Calendario para agendar fecha y hora. Pantallas de recordatorio y confirmación. Diseño gráfico, botones, menú, y demás de la parte visual para el usuario

Tecnologías en frontend

HTML, CSS, JAVASCRIPT, frameworks: React, Angular o Vue

Para el Backend que sería el funcionamiento de detrás de todo que es la parte que el usuario no ve, aquí nos encargamos de: Procesar solicitudes del frontend en este caso guardar la cita, Validar la disponibilidad de DRs y horarios, Manejar la autenticación y seguridad del usuario, Enviamos notificaciones y confirmaciones por correo.

Tecnologías según lo investigado

Node.js, Django, Spring Boot

Base de datos: MySQL, PostgreSQL, MongoDB

EL Frontend (cliente/usuario) y el Backend (servidor) se comunican por un flujo basado en API, HTTP y el modelo de request/ response.

El frontend (usuario selecciona una fecha y confirma la cita) envía un request HTTP (ejemplo un POST) ala API del backend (endpoints para interactuar con la lógica del sistema) y el backend (servidor) recibe el request y genera un response HTTP con datos estructurados (en JSON) que el Frontend usa para actualizar los datos de la página con la confirmación de la cita.

REST vs SOAP vs GraphQL ⚙️

Comparar estas opciones según formato de datos, flexibilidad, dificultad de implementación y uso actual, para entender cuándo conviene cada una.

Cómo completar la tabla:

- Usa términos concisos (p. ej., “JSON/XML” en formato de datos).
- En “nivel de flexibilidad”, piensa: ¿quién controla mejor lo que se envía/recibe, cliente o servidor?
- En “dificultad de implementación”, considera curva de aprendizaje, herramientas y mantenimiento.
- En “uso actual”, valora su presencia en proyectos modernos (Alta/Media/Baja).

Complete la siguiente tabla:

Tipo de API	Formato de datos usado	Nivel de flexibilidad	Dificultad de implementación	Uso actual (Alta / Media / Baja)
REST	JSON	Alta :endpoints simples	Facil: muy documentado	Mas usado
SOAP	XML	Baja: muy rigido Contratos extrictos	Complejo: estándares XML	Menos usado, casi no se usa por rigido
GraphQL	JSON	Muy alta: Cliente define el dato	Moderado: definir esquemas, acciones desde el request	Reciente y usado en apps modernas

Luego conteste:

¿Cuál considera más apropiada para una startup moderna que desarrolla un sistema de reservas en línea? ¿Por qué?

Según lo investigado lo más apropiado es REST ya que es muy simple y tiene estándares claros, es muy compatible con casi todos los lenguajes y frameworks. Pero al estudiar la GraphQL puede ser más complicada, pero nos da una diferenciación con API flexible y optima tizada para app móviles y web que es lo más común ahora.

4. Explorando APIs con Postman

Parte 1: Selección de la API

1. Busca una API pública gratuita (sin autenticación obligatoria o con token gratuito).

Por ejemplo podrías utilizar:

- [PokéAPI](#)

2. Escribe en tu documento:

- Nombre de la API
- `pokeapi`
- Breve descripción de lo que ofrece.

3. API GRATUITA Información completa sobre universo pokemon juegos, especies y mas

Parte 2: Configuración en Postman

1. Crea una **nueva colección** en Postman con el nombre de la API.
2. Agrega al menos **3 tipos de solicitudes**:
 - **GET** → obtener información (por ejemplo, lista o detalle de un elemento).
 - **POST** → enviar datos (por ejemplo, crear un recurso).
 - **PUT/PATCH o DELETE** → actualizar o eliminar un recurso (si la API lo permite).
3. Si la API usa **parámetros** o **tokens**, configura las variables necesarias en la pestaña **Environment**.

Parte 3: Ejecución y análisis

1. Ejecuta cada solicitud y verifica:
 - Código de estado (200, 404, 500, etc.).
 - Cuerpo de la respuesta (JSON o texto).
 - Headers relevantes (por ejemplo: Content-Type, Authorization).
2. Guarda las respuestas en tu colección.

GET pikachu

Código de estado 200 ok

JSON datos de estadísticas lista de juegos y sus URL.

HEADERS tenemos Host, User-Agent

SIMULACION POST

CODIGO DE ESTADO: 200 OK

JSON con los datos enviados

HEADERS content-type, content-length

SIMULACION PUT

CODIGO DE ESTADO: 200 OK

JSON CON DATOS ACTUALIZADOS

HEADERS Content-Type : application/json

SIMULACION Delete

CODIGO DE ESTADO: 200 OK

JSON confirmación de eliminación

HEADERS content-type: application/json

Parte 5: Reflexión final ☐

En 1 o 2 párrafos, responde:

- ¿Qué aprendiste sobre el funcionamiento de las APIs?
- ¿Cómo te ayudó Postman a entender la comunicación entre cliente y servidor?

Las API's funcionan como puente entre aplicaciones, lo que nos permite que un cliente en este caso postman solicite, envíe, datos a un servidor mediante métodos HTTP. Cada endpoint expone un recurso y la respuesta viene en formato JSON. Algunas API's son únicamente de lectura y otras permiten todas las operaciones como crear, actualizar, y eliminar datos.

Comunicación entre cliente y servidor con Postman podemos visualizar esta comunicación de manera practica, enviando solicitudes, ver los código de estado, los headers y Body se comprueba como funciona la fluidez de la información q ocurre cuando hacemos una petición. Además poder modificar las colecciones y simular escenarios de interacción con un API podemos ver como cada solicitud estructuralmente la comunicación cliente – servidor.